From Waveform to Score: Pipeline for Monophonic Pitch Tracking and MIDI-Based Music Transcription

Sarvesh Satish
Heartfulness International School
Sarveshsatish09@gmail.com, Grade 12

Abstract—This paper presents an end-to-end pipeline for converting a short monophonic trumpet recording into symbolic musical notation. The pipeline combines a static Fast Fourier Transform (FFT) spectrum, a Short-Time Fourier Transform (STFT) spectrogram, and the pYIN algorithm for frame-wise fundamental-frequency (f_0) estimation, together with Mel-frequency cepstral coefficients (MFCCs), spectral centroid, spectral bandwidth, and chromagram features for timbral and harmonic characterization. The continuous f_0 contour is quantized into fifteen discrete note events, exported as a MIDI file, and rendered into engraved sheet music using a commercial notation editor. While the pipeline succeeds in producing a complete, playable transcription, closer inspection of the intermediate features reveals two systematic error sources: an apparent octave-tracking error in the pYIN contour relative to the note range implied by the quantized output, and misclassification of a 1.83-second decay/noise tail as a single sustained note, corroborated by a sharp rise in spectral centroid ($\sim$2,000 Hz to $\sim$6,000 Hz) and bandwidth ($\sim$1,000 Hz to $\sim$3,500 Hz) in that interval. These findings are used to motivate concrete, targeted improvements — a voicing/energy consistency check and an octave-correction step — rather than speculative architectural additions.

Index Terms—audio signal processing, pitch detection, pYIN, Short-Time Fourier Transform, music information retrieval, automatic music transcription, MIDI

I. INTRODUCTION

Automatic music transcription — the conversion of a raw audio recording into a symbolic representation such as sheet music — sits at the boundary between continuous signal processing and discrete musical structure. A recorded performance is a continuous waveform, but notation requires committing to discrete pitches, onsets, and durations. The process of making that commitment, rather than the frequency-domain transforms that precede it, is where most of the practical difficulty in this study turned out to lie.

This work was carried out as an independent study with two goals: first, to implement a complete pipeline from raw audio to notated output using standard open-source signal-processing tools; and second, to examine *where* and *why* that pipeline produces unreliable output, rather than treating a plausible-looking transcription as evidence of correctness. The second goal motivates the error analysis in Section IV, which is the primary contribution of this report.

Concretely, the objectives were to: (1) apply the Fast Fourier Transform (FFT) and Short-Time Fourier Transform (STFT) to obtain static and time-resolved spectral representations of a monophonic recording; (2) identify musical notes using both static peak-picking and the pYIN dynamic pitch-tracking algorithm [5]; (3) extract complementary timbral and harmonic features (MFCCs, spectral centroid, spectral bandwidth, chromagram); (4) quantize the estimated pitch contour into discrete note events and export them as a MIDI file; and (5) evaluate the resulting transcription against the intermediate spectral evidence to identify systematic failure modes.

II. METHODOLOGY

The pipeline was implemented in Python using `librosa` [1] for audio feature extraction, `SciPy` [2] for the FFT and peak detection, `Matplotlib` [3] for visualization, and `mido` [9] for MIDI file generation.

A. Audio Data Acquisition

A single-instrument (trumpet) recording, obtained via `librosa.ex()`, was loaded at a sampling rate $sr = 22{,}050$ Hz, giving a Nyquist frequency of 11,025 Hz — sufficient to capture the fundamental and lower harmonics of the instrument's playing range.

B. Static Spectral Analysis (FFT)

For a discrete signal $y[n]$ of length N, the discrete Fourier transform, computed efficiently via the FFT [4], is

$$Y[k] = \sum_{n=0}^{N-1} y[n]\, e^{-i2\pi kn/N}, \qquad k = 0, \ldots, N-1, \tag{1}$$

where $Y[k]$ corresponds to frequency $f_k = k \cdot sr/N$. Since $y[n]$ is real, only the first $N/2$ bins are independent; the magnitude spectrum used for peak detection is

$$M[k] = \frac{2}{N} |Y[k]|, \qquad k = 0, \ldots, N/2 - 1. \tag{2}$$

Peaks in $M[k]$ were located with a dynamic threshold of 10% of the maximum magnitude and mapped to note names via `librosa.hz_to_note()`.

C. Time-Resolved Spectral Analysis (STFT)

To resolve how the spectrum evolves in time, the STFT slides an N_{fft}-sample window across the signal in hops of H samples:

$$X[k,m] = \sum_{n=0}^{N_{\text{fft}}-1} y[n+mH]\, w[n]\, e^{-i2\pi kn/N_{\text{fft}}}, \tag{3}$$

with $N_{\text{fft}} = 2048$, $H = 512$. The resulting amplitude spectrogram is converted to decibels,

$$X_{\text{dB}}[k,m] = 20\log_{10}\left(\frac{|X[k,m]|}{\max|X|}\right). \tag{4}$$

D. Dynamic Pitch Tracking

The pYIN algorithm [5], a probabilistic extension of YIN [6], was used to estimate a frame-wise fundamental frequency $f_0[m]$ together with a voicing flag, giving a continuous pitch contour rather than the single static spectrum obtained from Eq. (2).

E. Timbral and Harmonic Feature Extraction

Four additional features were computed per STFT frame. MFCCs (13 coefficients) [7] summarize the spectral envelope. Spectral centroid, a proxy for perceptual brightness, is

$$C[m] = \frac{\sum_k f_k\,|X[k,m]|}{\sum_k |X[k,m]|}, \tag{5}$$

and spectral bandwidth, the energy-weighted spread around the centroid, is

$$B[m] = \sqrt{\frac{\sum_k (f_k - C[m])^2\,|X[k,m]|}{\sum_k |X[k,m]|}}. \tag{6}$$

The chromagram folds spectral energy into twelve octave-independent pitch classes.

F. Note Quantization and MIDI Export

Consecutive voiced frames mapped to the same note name were grouped into note events with a start time, end time, and duration; groups shorter than a minimum-duration threshold were discarded as likely tracking artifacts. The resulting sequence was written to a standard MIDI file using `mido`, with a fixed tempo of 120 BPM and a uniform velocity of 100 (i.e., dynamics were not estimated from signal amplitude in this implementation).

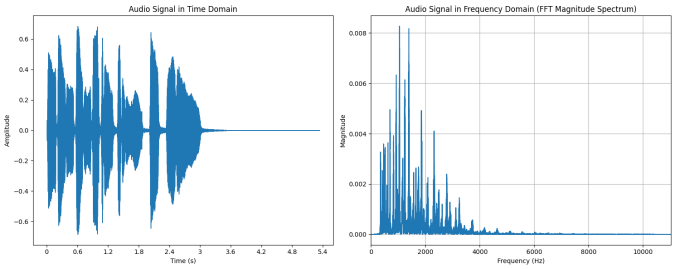


Fig. 1. Time-domain waveform (left) and FFT magnitude spectrum, Eq. (2) (right).

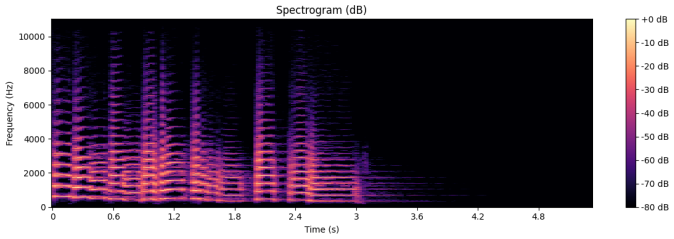


Fig. 2. Decibel-scaled spectrogram, Eq. (4).

G. From MIDI to Notation

The MIDI file was imported into Garageband [10], which interprets note-on/note-off events directly to render pitches, durations, beaming, and rests on a staff. Minor manual adjustment (beam grouping, triplet bracketing) was applied in the notation editor; no automatic quantization-correction or key/time-signature inference was used beyond Garageband's defaults.

III. RESULTS AND ANALYSIS

A. Static and Time-Resolved Spectra

Fig. 1 shows the time-domain waveform and the FFT magnitude spectrum (Eq. (2)). Distinct amplitude pulses are visible for approximately the first three seconds, followed by a low-amplitude tail. The magnitude spectrum shows a clear peak at 348.75 Hz (F4), along with G#4, A#4, C5, and D5, consistent with a melodic phrase confined to a roughly one-octave range — but the static FFT provides no information about the order or timing of these pitches.

The STFT spectrogram (Fig. 2) resolves this temporal ambiguity: vertical striations recur roughly every 0.2–0.3 s for the first three seconds, consistent with the note onsets in Table 1. After approximately 3 s, the dense harmonic bands give way to broadband, noise-like energy — a transition that is central to the error identified in Section IV.

B. Pitch Contour

Fig. 3 overlays the pYIN f_0 estimate on the spectrogram. The contour remains close to a narrow 400–500 Hz band for most of the clip, with only small deviations, despite the wider pitch

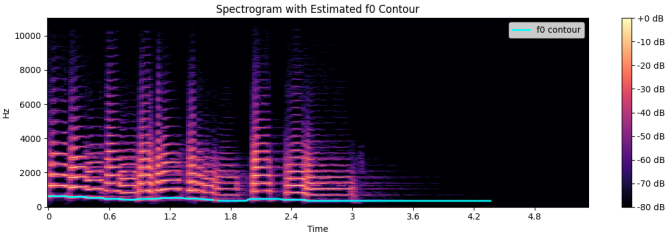


Fig. 3. Spectrogram with pYIN f_0 contour overlaid in cyan.

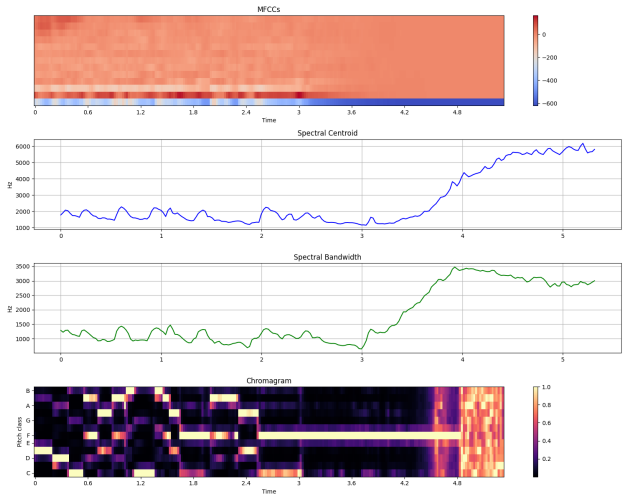


Fig. 4. Extracted MFCCs, spectral centroid (Eq. (5)), spectral bandwidth (Eq. (6)), and chromagram.

range implied by both the static FFT peaks and the final quantized note sequence (Section IV-A).

C. Timbral and Harmonic Features

Fig. 4 shows the extracted MFCCs, spectral centroid, spectral bandwidth, and chromagram. Three observations are relevant to the transcription result: (i) the MFCC map is stable for approximately the first 3 s and flattens thereafter; (ii) the spectral centroid remains within 1,000–2,300 Hz during the melodic portion of the clip, then rises sharply after ∼3.6 s to roughly 6,000 Hz; and (iii) the spectral bandwidth follows the same pattern, widening from under 1,500 Hz to approximately 3,500 Hz over the same interval. A concurrent, sharp increase in both centroid and bandwidth is not typical of a sustained instrumental pitch and is a stronger indicator of broadband noise — consistent with instrument decay and breath noise rather than a musical note.

D. Quantized Notes and Transcription Output

Table 1 lists the fifteen note events recovered from the f_0 contour after quantization. The exported MIDI file was rendered as sheet music in Garageband (Fig. 5), including triplet groupings and accidentals recovered directly from the note-duration data.

TABLE I
Quantized Note Events Extracted from the f_0 Contour

Note	Start (s)	End (s)	Dur. (s)
D#5	0.00	0.19	0.19
D5	0.19	0.39	0.21
C5	0.39	0.56	0.16
A#4	0.56	0.74	0.19
G#4	0.74	0.88	0.14
A#4	0.88	1.04	0.16
B4	1.04	1.14	0.09
C5	1.14	1.35	0.21
B4	1.35	1.46	0.12
A#4	1.46	1.56	0.09
G#4	1.56	1.65	0.09
F4	1.67	1.97	0.30
A#4	2.00	2.30	0.30
G#4	2.32	2.55	0.23
F4	2.55	4.39	1.83

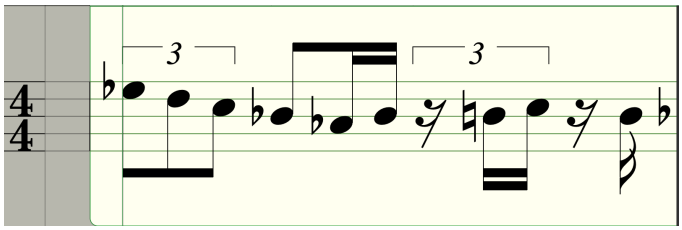


Fig. 5. Engraved excerpt produced in Garageband from the exported MIDI file, corresponding to Table 1.

IV. LIMITATIONS AND ERROR ANALYSIS

Three specific, evidence-backed limitations were identified in the transcription output; these are distinguished here from generic, textbook-level caveats about audio analysis.

A. Apparent Octave-Tracking Error

The pYIN contour in Fig. 3 remains close to a single 400–500 Hz band for most of the recording, yet Table 1 reports notes spanning D#5 down to F4 — more than an octave. The most plausible explanation is that pYIN locked onto a subharmonic of the true pitch for extended stretches, a known failure mode when a lower harmonic carries comparable energy to the fundamental [6]. This was not independently verified against a ground-truth score, and is reported as an open limitation of the result rather than a resolved finding.

B. Borderline-Duration Notes

Two notes in Table 1 (B4 at 1.04–1.14 s; G#4 at 1.56–1.65 s) last only 0.09 s. Given that the minimum-duration filter already discards the shortest, most clearly spurious segments, these remaining short events lie close enough to the filter threshold that they cannot be confidently distinguished from brief pitch-tracker instability rather than deliberate, played notes.

C. Decay/Noise Misclassified as a Sustained Note

The final entry in Table 1 — F4, 2.55–4.39 s, duration 1.83 s — coincides exactly with the interval in which spectral centroid and bandwidth both spike sharply (Section III-C) and the spectrogram shows broadband rather than harmonic energy (Section III-A). This strongly suggests the segment is instrument decay and breath noise rather than a genuinely sustained pitch; because pYIN nonetheless reported a low, quasi-stable f_0 value throughout, the note-grouping step merged it into a single long note. This is judged to be the least reliable entry in the transcription.

D. Untuned Parameters

The FFT peak-detection threshold (10% of maximum magnitude) and STFT settings ($N_{\text{fft}} = 2048$, $H = 512$) were fixed a priori rather than tuned against a validation set. Different settings, particularly a smaller hop length, would likely shift the exact note boundaries in Table 1 without necessarily changing the overall melodic shape.

V. FUTURE WORK

The following directions follow directly from Section IV, rather than being independent extensions:

1. **Voicing/energy consistency check.** Cross-referencing f_0 against spectral bandwidth (Eq. (6)) could flag segments like the decay tail in Section IV-C, where an unusually wide bandwidth suggests the "note" is not a genuine pitch.
2. **Octave-error correction.** A post-processing step comparing candidate f_0 values against harmonic peaks in the static FFT spectrum (Eq. (2)) could help disambiguate a fundamental from its subharmonic, addressing Section IV-A.
3. **Confidence-weighted duration filtering.** Replacing the fixed minimum-duration cutoff with one weighted by pYIN's own voicing probability would directly target the borderline notes in Section IV-B.
4. **Evaluation against ground truth.** Testing the pipeline on a recording with a known, written score would allow quantitative accuracy measurement in place of the qualitative, feature-based validation used in this report.

VI. CONCLUSION

This report implemented and evaluated a complete pipeline for converting a monophonic recording into notated music, using the FFT, STFT, and pYIN algorithm for pitch estimation, MFCCs and related descriptors for timbral characterization, and a custom quantization step to produce a MIDI file rendered as sheet music. The pipeline produced a complete, playable transcription (Fig. 5), but a closer, feature-level examination of the intermediate outputs — rather than the final score alone — revealed that not all fifteen recovered notes are equally trustworthy. In particular, the f_0 contour is consistent with a probable octave-tracking error, and the final, longest note in the transcription is better explained by instrument decay and noise than by a sustained pitch, based on independent evidence from spectral centroid and bandwidth. These findings suggest that evaluating an automatic transcription pipeline solely by whether its output "looks like music" is insufficient; cross-checking pitch estimates against complementary spectral features is a low-cost way to identify likely errors, and is the basis for the targeted improvements proposed in Section V.

ACKNOWLEDGMENT

The author gratefully acknowledges Dr. Kiran Siripuri, Symbiosis Institute of Technology,Hyderabad, for his valuable guidance and Dr. Sumit Soman, Senior Member, IEEE for his mentorship throughout this work.

REFERENCES

[1] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 18–24.

[2] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[3] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[4] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.

[5] M. Mauch and S. Dixon, "pYIN: A fundamental frequency estimator using probabilistic threshold distributions," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, 2014, pp. 659–663.

[6] A. de Cheveigné and H. Kawahara, "YIN, a fundamental frequency estimator for speech and music," *J. Acoust. Soc. Am.*, vol. 111, no. 4, pp. 1917–1930, 2002.

[7] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.

[8] MIDI Manufacturers Association, *The Complete MIDI 1.0 Detailed Specification*. La Habra, CA: MMA, 1996.

[9] "mido: MIDI Objects for Python," [Online]. Available: `https://mido.readthedocs.io/`

[10] Garageband , "Apple Inc. GarageBand, version 10.4.11, 2026. Apple App Store." [Online]. Available: `https://apple.com/`

[11] M. Müller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Cham, Switzerland: Springer, 2015.